Reinforcement Learning Teaches a Small Language Model to Use an External Notepad Better than Its Own Context Window

Sten Rüdiger
https://github.com/sr-networks/clbench-memory-rl

July 2026

Abstract

Agentic benchmarks of continual learning have reported a puzzling result [1]: giving a language model an external *notepad* for carrying information across steps does not reach the accuracy of plain in-context learning (ICL), in which the entire interaction history simply remains in the prompt. We ask whether this deficit is a property of the notepad interface or of the model's untrained *policy* for using it. On a long-horizon continual-monitoring task (30 sequential scans of a partially observable radio band), we train Qwen3-1.7B with GRPO under a reward that pays exclusively for recalling information that is no longer observable, and that is orthogonal to single-step task skill by construction. We find that (i) full-history ICL is strongest only in the first few steps and then degrades as the transcript grows, ending barely above a scripted no-memory floor; (ii) the same base model equipped with a notepad already surpasses ICL from step 6 onward *without any training*; and (iii) reinforcement learning further lifts late-episode accuracy from 0.45 to 0.61 occupied-IoU—replicated in direction across four independent runs—while first-step accuracy, where the notepad is empty, remains unchanged ($0.216 \rightarrow 0.217$), confirming that the gain is memory use rather than task skill. The reported notepad–ICL gap is therefore regime-dependent: over long horizons a trained (and even an untrained) external memory dominates in-context history.

1 Introduction

Language-model agents that act over long horizons must carry information across steps. Two interface designs dominate practice. Under *in-context learning* (ICL), nothing is discarded: every past observation, action, and result stays in the prompt, and the model re-reads its own history. Under an *external memory* [4], the context is windowed to the current step, and the agent maintains a small, editable buffer—a *notepad*—that is the only information surviving from one step to the next.

A priori, the notepad appears preferable: it is a compressed, curated summary under the agent's control, it keeps the prompt short, and it does not bury salient facts under many turns of transcript. Yet the Continual Learning Bench (CLBench) [1], from which our task is derived, reported the opposite ordering: notepad-equipped agents did not reach the accuracy of full-history ICL. This is consistent with a simple hypothesis that has, to our knowledge, not been isolated experimentally: pretrained models have extensive implicit training in exploiting context, but essentially none in *maintaining* an external memory. The interface is not worse; the policy for using it is untrained.

This paper tests that hypothesis with a controlled reinforcement-learning (RL) study on a small model, Qwen3-1.7B [5]. We pose one question:

> *Can RL train a small model to use an external notepad well enough to match or exceed full-history ICL—without improving the model's single-step task competence?*

The italicized rider is essential. An RL run that improves a memory-dependent score may have (a) memorized task content into the weights, (b) improved single-step skill, or (c) genuinely improved the carrying of information forward. Only (c) is "training memory." Our experimental design excludes (a) and (b) by construction, by penalty, and by measurement (Section 3).

Our contributions are: (1) a memory-only reward for a long-horizon monitoring task, red-teamed against blanket-reporting and weight-memorization exploits before training; (2) a five-condition comparison—scripted no-memory floor, full-history ICL, untrained notepad, RL-trained notepad, scripted perfect-memory ceiling—aligned by step position within the episode; and (3) evidence that the published notepad–ICL gap reverses over long horizons: ICL degrades as the transcript grows, consistent with known long-context limitations [2], while the trained notepad accuracy rises and holds.

2 Task and Memory Interfaces

Blind spectrum monitoring. The task is derived from the blind spectrum monitoring family of CLBench. An agent monitors one fixed radio-frequency band containing 11–14 persistent transmitter channels. It receives n=30 sequential scans of the same band. In any single scan, only ∼3 transmitters are actively emitting; dormant transmitters still *occupy* the band. After each scan the agent submits a report listing every frequency region it believes to be persistently occupied (center frequency and bandwidth per region).

Detector errors. Following the benchmark's data-generating process, the scan data shown to the agent are deliberately unreliable. Each actively emitting transmitter is missed with probability 0.15, so even an active channel may produce no peak. When a peak is reported, its parameters are noisy estimates: the frequency carries Gaussian error (1 MHz standard deviation), the width is perturbed by about 15%, the power by 5 dBm, and the transmitter's true center itself drifts by up to ±0.5 MHz between scans. Scans also contain clutter that corresponds to no persistent transmitter: with probability 0.2 a spurious noise peak appears at a random frequency, and with probability 0.2 a scan contains one to three narrow interference spikes. The agent therefore cannot simply copy single detections into its report; it must aggregate repeated noisy sightings into stable channel estimates and distinguish persistent transmitters from clutter.

Score. The per-scan score is the occupied-spectrum intersection-over-union (occ-IoU) between the reported regions and the true persistent transmitter set,

$$\text{occ} \;=\; \frac{|R \cap T|}{|R \cup T|} \in [0, 1], \tag{1}$$

where R is the union of reported frequency intervals and T the union of true occupied intervals, both measured in MHz. Because only a minority of channels is visible per scan, a high occ requires reporting channels observed in *earlier* scans that are currently silent—i.e., memory.

Scripted bounds: no-memory floor and perfect-memory ceiling. Two scripted oracle agents, replayed through the real task engine on the same bands, bracket the comparison. The *no-memory* agent, on every scan, matches each visible peak against the hidden ground truth, identifies which true channels produced the current detections, and reports exactly those channels with their true center frequencies and widths. It thus resolves the detector noise perfectly—something no real agent can do, since a model must estimate centers and widths from the noisy peaks themselves—but it remembers nothing across scans. Its score, occ ≈ 0.26, is therefore not a weak baseline but an upper bound for any memoryless policy (the per-variant

floors $a_0 \in \{0.2082, 0.2141, 0.2368\}$ enter the reward in Section 3). The *perfect-memory* agent is identical except that its set of identified channels persists across the whole episode: it reports every channel it has ever seen. This is the ceiling of the comparison—total recall combined with perfect perception. It is necessarily low early (channels that have not yet appeared cannot be known to any agent: 0.60 over scans 1–5) and approaches occ ≈ 0.99 by the final scans as the episode uncovers the band. Every model condition lives between these two bounds.

Memory interfaces. We compare two designs holding the base model fixed:

- **ICL (full history).** Nothing is wiped; all past scans, reports, and tool results remain in the prompt. No notepad exists.
- **Notepad (windowed context).** The context contains only the current scan. The agent has `notepad_read`/`notepad_write` tools and a ∼4000-character buffer it may overwrite each turn; this buffer is the only information crossing scan boundaries. Maintaining it is the agent's own job.

The notepad system prompt teaches the *procedure* (read the notepad, merge the current peaks, write back the complete running set, report the full persistent set); it does not contain any band content.

3 A Memory-Only Reward

RL on a memory-dependent score invites two families of confound: improving the memory-free part of the task, and shortcut policies that fake coverage. The reward is designed so that neither pays.

Per-scan memory score. For scans $t = 2, \ldots, n$ let the *recallable set* D_t be the channels that are invisible at scan t but were visible in some earlier scan. The per-scan memory score is a Tversky overlap (α=β=1) between the report and D_t:

$$\text{SCAN\_DORM}_t \;=\; \frac{\text{cov}(R_t, D_t)}{\text{cov}(R_t, D_t) + \text{FP}_t + \text{FN}_t}, \tag{2}$$

All three quantities are spectrum areas, measured in MHz along the frequency axis (in the implementation the band is discretized into 0.5-MHz bins and areas are bin counts). Let R_t denote the union of the frequency intervals the agent reports at scan t, let D_t denote (by slight abuse of notation) also the union of the frequency intervals of the recallable channels, and let A_t denote the union of the intervals of the channels that are visible now *or* were visible in some earlier scan. Then $\text{cov}(R_t, D_t) = |R_t \cap D_t|$ is the *covered recallable area*: the part of the spectrum belonging to currently-dormant, previously-seen channels that the report still marks as occupied. $\text{FN}_t = |D_t| - |R_t \cap D_t|$ is the recallable area the report misses, and—critically—$\text{FP}_t = |R_t \setminus A_t|$ is the reported area on spectrum the agent has *never seen occupied*. Currently visible channels are excluded from both numerator and penalties: reporting what is on screen is neither rewarded nor punished. This memory score is therefore orthogonal to single-step skill by construction, and it cannot be earned without information crossing scan boundaries.

Episode structure. A training episode consists of the 30 scans of one band. At each scan the model receives only the current scan's peak list together with the current contents of its notepad; the transcript of all earlier scans has been wiped from the context. The model may rewrite the notepad and then submits its occupancy report, and whatever it wrote to the notepad is the only information that survives into the next scan. When the episode ends, the whole rollout receives a single scalar reward.

Episode reward. The reward combines the mean memory score with four penalties:

$$S = 3\left(\overline{\text{dorm}} - p_{\text{anchor}} - p_{\text{carpet}} - p_{\text{wmax}} - p_{\text{complete}}\right), \tag{3}$$

where $\overline{\text{dorm}}$ is the mean of SCAN_DORM_t over the scans actually played. Each penalty targets one specific way of inflating the score without using memory, and each is exactly zero on honest play:

- *Anchor penalty.* $p_{\text{anchor}} = \max(0,\ \text{anchor} - a_0 - 0.10)$, where anchor is the mean occ over scans 1–2. On the first two scans almost nothing is recallable yet, so a score far above the scripted memoryless floor a_0 of that band variant cannot come from memory; any surplus beyond a margin of 0.10 is subtracted.
- *Area penalty.* $p_{\text{carpet}} = 4.0 \cdot \max(0,\ \overline{\text{rarea}} - 1.15)$, where rarea is the total reported area divided by the true occupied area of the band. This penalty punishes the blanket strategy of declaring large parts of the band occupied instead of remembering channels: it activates as soon as the model reports more than 115% of the truly occupied area.
- *Width penalty.* $p_{\text{wmax}} = 1.0 \cdot \max(0,\ \overline{\text{wmax}} - 1.5)$, where wmax is the width of the widest reported region divided by the width of the widest true channel. This penalty punishes the same blanket strategy executed as one single, very wide region, which would inflate the covered area faster than the area penalty alone can tax it.
- *Completion penalty.* $p_{\text{complete}} = 1.5 \cdot \max(0,\ 30 - n_{\text{played}} - 2)/30$, where n_{played} is the number of scans the model completed. Because $\overline{\text{dorm}}$ averages only over played scans, a policy could otherwise raise its mean by ending the episode early, right after its strongest scans. Completing at least 28 of the 30 scans makes this penalty exactly zero.

An episode with zero completed scans (a formatting failure on the first turn) receives a flat $S = -0.2$.

Adversarial audit. Each penalty corresponds to an exploit we simulated offline before training: declaring the whole band occupied is caught by the area penalty; doing the same with one very wide region by the width penalty; behaving honestly on scans 1–2 to avoid the anchor penalty and blanketing afterwards by the per-scan area and width statistics; and ending the episode early after a few strong scans by the completion penalty. A further exploit—memorizing the fixed channel grid into the weights—is unprofitable by the definition of the memory score itself: false positives are measured against spectrum the agent has never seen occupied, which an honest memory policy never reports but a memorized grid must. Empirically, all four penalties are ≈ 0 at epoch 0 in every run, so on honest play the reward is exactly $3\,\overline{\text{dorm}}$, a quantity that can be earned only through memory. We deliberately never reward the late-minus-early improvement itself: rewarding an improvement delta would invite deliberately weak early scans.

Weight-memorization detector. On scan 1 the notepad is empty, so any trained-versus-untrained gain there must be band knowledge stored in the weights rather than memory use. Scan-1 occ is therefore measured for every run as a no-memory discriminator; the result (Section 5) is flat to within $+0.001$.

4 Experimental Setup

Training. Base model Qwen3-1.7B [5]; GRPO [3] with LoRA adapters on a managed reinforcement fine-tuning service. We ran four replicate runs with identical configuration and different seeds: 24 training bands, learning rate 2×10^{-4}, 5 epochs, 8 candidate rollouts per

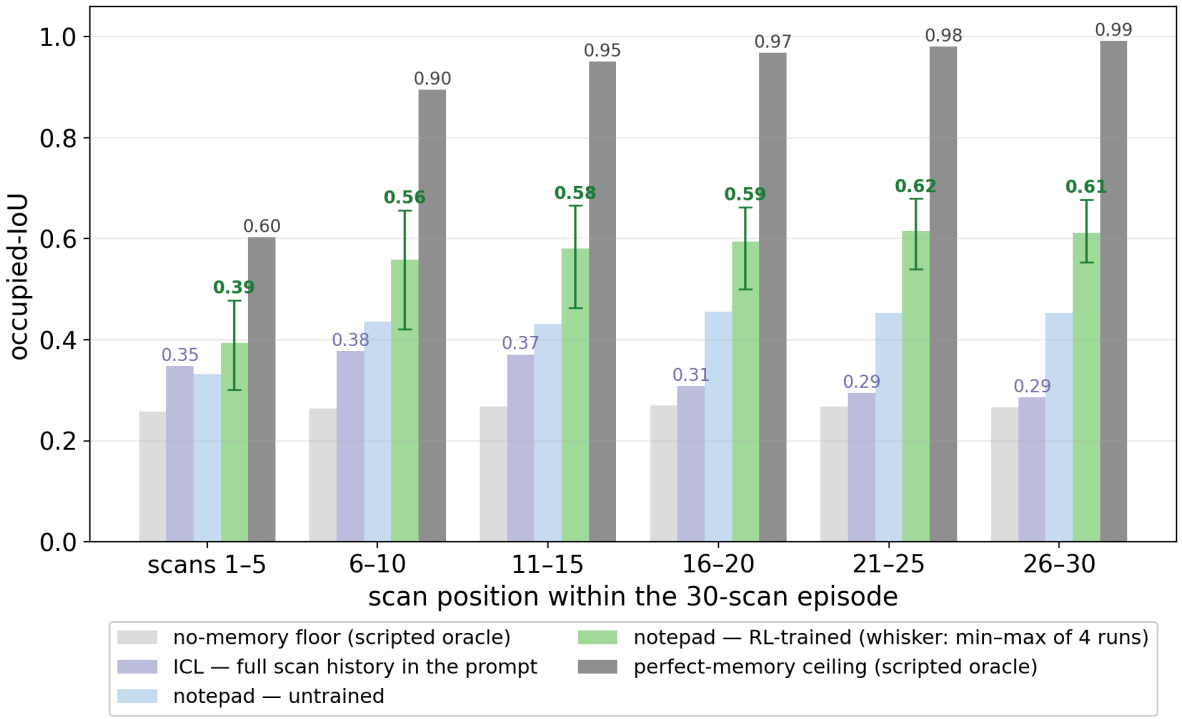

Figure 1: Occupied-IoU by scan position within the 30-scan episode, for the five conditions. Full-history ICL (purple) peaks at scans 6–10 and then degrades as the transcript grows, ending barely above the scripted no-memory floor (light grey). The untrained notepad (light blue) surpasses ICL from scan 6 onward; RL (green; whiskers give the min–max of the four replicate runs) lifts late-episode accuracy to ≈0.61. The scripted perfect-memory ceiling (dark grey) rises from 0.60 to 0.99 as the episode uncovers the band.

GRPO group, sampling temperature 1.2, and a 16,384-token per-turn output cap. Rewards follow Eq. (3). Per epoch and run, evaluation uses 192 rollouts. "Trained" denotes the final epoch (ep4) and "untrained" the same runs at epoch 0; "pooled" means that the episodes of all four replicate runs are aggregated and treated as one sample (4×192 episodes per epoch).

Conditions. The central comparison aligns five conditions by scan position within the episode:

condition	policy	memory	rollouts
no-mem	scripted perfect memoryless agent	none (floor)	deterministic replay
ICL	untrained base, full history in prompt	in-context	288
notepad-untrained	untrained base with notepad tools	notepad	4×192 (ep0, pooled)
notepad-trained	same four runs after RL	notepad	4×192 (ep4, pooled)
perfect-mem	same scripted agent with total recall	perfect (ceiling)	deterministic replay

ICL-versus-notepad-untrained is a pure comparison of memory *interface* (same weights); notepad-untrained-versus-trained isolates the *training* effect. Scores are pooled into six bins of five consecutive scans.

5 Results

Figure 1 and Table 1 contain the main result.

ICL degrades over the episode. Full-history ICL leads only in the first bin (0.348 vs. 0.332 for the untrained notepad), peaks at scans 6–10 (0.378), and then falls monotonically—0.371,

Table 1: Occupied-IoU by scan-position bin (higher is better; bold marks the best model condition). Notepad columns pool four replicate runs; the bracketed range is the min–max of the four per-run means at ep4. The perfect-mem column is the scripted total-recall ceiling. n is the number of trained-condition scan observations pooled per bin. Episode-level bootstrap 95% confidence intervals of the pooled bin means (2000 resamples of whole episodes) are no wider than ±0.011 for the notepad conditions and ±0.022 for any model condition (±0.046 for the 24-episode scripted ceiling); beyond the first bin the five conditions' intervals are pairwise non-overlapping. Full CIs ship with the data (Data availability).

scans	no-mem	ICL	notepad ep0	notepad ep4	perfect-mem	n_{ep4}
1–5	0.257	**0.348**	0.332	0.394 [0.301–0.478]	0.603	3840
6–10	0.264	0.378	0.436	**0.559** [0.421–0.656]	0.895	3840
11–15	0.268	0.371	0.431	**0.581** [0.463–0.666]	0.950	3825
16–20	0.270	0.308	0.455	**0.595** [0.501–0.662]	0.968	3808
21–25	0.268	0.294	0.454	**0.615** [0.540–0.679]	0.980	3737
26–30	0.266	0.286	0.453	**0.612** [0.553–0.678]	0.991	3171

0.308, 0.294, 0.286—although strictly more information is present at every scan. By the final bin it is barely above the no-memory floor (0.266), and its late-half mean (scans 16–30) is 0.297. This is the long-context failure mode at small scale: facts buried twenty-plus scans deep in a growing transcript effectively stop being used, consistent with positional-use limitations of long contexts [2]. The decline is not a context-window artifact: 61% of ICL episodes complete all 30 scans (final transcripts average ≈30k tokens), the decline is unchanged when the analysis is restricted to complete episodes, and it persists in the subset of complete episodes whose entire transcript remained under 30k tokens (0.344 → 0.258 from the first bin to the last, ending at the no-memory floor). The information is present in the window throughout; the model stops using it.

The untrained notepad already beats ICL beyond scan 5. With identical weights, windowing the context and providing the notepad yields ≈0.45 occ throughout the late half—+0.16 over ICL—before any training. A short, curated summary pinned at a fixed position does not rot the way raw history does. This alone reframes the benchmark finding that motivated the study: the notepad interface is not inherently weaker; over long horizons it is inherently more robust.

RL lifts the notepad to a decisive win. Training raises late-half occ from 0.454 to 0.607 (pooled), a +0.31 margin over ICL and roughly a doubling of the headroom above the no-memory floor. The direction of the effect replicated in four of four runs (complete-episode mean recallable-coverage +0.041, +0.101, +0.181, +0.182); the magnitude varies by seed, which the whiskers in Figure 1 report rather than hide. Against the perfect-memory ceiling (late-half ≈0.98), the achievable memory headroom in the late half spans about 0.71 occ: the trained notepad captures ≈48% of it, the untrained notepad ≈26%, and ICL ≈4%.

The gain is memory, not skill. Scan-1 occ, measured on an empty notepad, moves by +0.0004 to +0.0011 across the four runs; averaged over all episodes of the four runs it is 0.216 before training and 0.217 after. The trained model is thus not better at the task when it has nothing to remember. Combined with the construction of the memory score (visible channels carry no reward), the anchor penalty, and the remaining penalties, every improvement in Table 1 is attributable to carrying information forward.

Training ICL does not fix the collapse. For completeness we also trained an ICL arm under a dense per-scan accuracy reward (4096-token per-turn cap). It learned to degrade somewhat less over the long context (mean occ 0.344 → 0.363), but its late-minus-early occ

remained negative in every epoch: it never learned to accumulate. The degradation of in-context memory over long horizons appears to be the binding constraint at this model scale, not an artifact of the untrained comparison.

6 Limitations

Survivorship in late bins. At ep4, episode completion across the four runs is 100%, 97%, 60%, and 90%; one run truncates episodes early. Late bins therefore average only scans actually played (n falls from 3840 to 3171), a disclosed survivorship bias. *ICL condition.* The ICL bar is the untrained base model under a 4096-token per-turn cap; the trained-ICL check above mitigates but does not remove the asymmetry. In the ICL condition 61% of episodes complete all 30 scans; the decline is unchanged when restricted to complete episodes (Section 5). *Scale and scope.* One base model (Qwen3-1.7B), one optimizer (GRPO), one task family; learning rate, temperature, and group size were not swept. *Benchmark relation.* This is a re-implementation of the CLBench task family as an RL environment, not the benchmark's exact harness; the contribution is the mechanism and its trainability, not a leaderboard number.

7 Conclusion

This study set out to explain a counterintuitive benchmark finding: language-model agents given an external notepad performed worse than agents that simply kept their entire history in the prompt. Our results indicate that this ordering is a property of short horizons and untrained policies, not of the notepad interface itself.

Over a thirty-scan monitoring episode, keeping the full history in the prompt is the best strategy only at the beginning. As the transcript grows, the model becomes progressively worse at using the information buried inside it, and by the end of the episode its accuracy is barely above that of an agent with no memory at all. The external notepad—a short summary that the agent itself maintains, and that appears at a fixed place in an otherwise short prompt—does not suffer from this degradation. Even without any training, the same base model scores higher with the notepad than with its full history from the sixth scan onward.

Reinforcement learning then makes the notepad policy substantially better. Trained under a reward that pays only for recalling channels that are currently invisible, and that was audited in advance against blanket-reporting, weight-memorization, and early-termination exploits, the model raises its late-episode accuracy from 0.45 to 0.61, and the direction of this improvement replicates in all four independent runs. Crucially, its accuracy on the first scan—where the notepad is still empty and memory cannot help—does not change. The model did not become better at the monitoring task; it became better at writing down what it would later need and reading it back.

For small models acting over long horizons, the practical implication is direct: rather than accumulating an ever longer history in the prompt, train the model to maintain an external memory. How far this recipe extends—to larger base models, to other optimizers, and to task families whose memory content is harder to discover and to apply—is the natural next question.

Data availability

All numbers in Table 1 and Figure 1 are recomputable from raw data. The repository at `https://github.com/sr-networks/clbench-memory-rl` contains the raw per-episode, per-scan traces of all five conditions (55,140 rows), a script that regenerates the figure and the binned table—including the bootstrap confidence intervals—from those traces alone, the executed reward code verbatim, the scripted no-memory and perfect-memory agents, the task description,

and the complete run registry. In the repository, runs are identified by opaque training-job IDs, so each published number is traceable to its source run.

References

[1] P. Asawa, C. M. Glaze, G. Orlanski, R. Ramakrishnan, B. Xu, A. Biswal, V. S. Chen, F. Sala, M. Zaharia, and J. E. Gonzalez. Continual Learning Bench: Evaluating frontier AI systems in real-world stateful environments. *arXiv preprint arXiv:2606.05661*, 2026.

[2] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[3] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[4] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[5] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.